

- 1. A feladatban bekérünk egy számot és eldöntjük, hogy prím-e? A feladatra 7 megoldást adunk, ahol is a kevésbé hatékony felől haladunk az egyre hatékonyabb megoldás felé */**

```
# include <iostream>
# include <cmath> // a matematikai függvények szerepelnek ebben az állományban
using namespace std;
int main( )
{int i,szam,osztok=0;
    cout<<"Adjon meg egy számot";
    cin>>szam;
    /* for(i=2;i<szam-1;i++)                //1. megoldás
        if(szam%i==0) osztok++;
    if(osztok==0) cout<<"prim";
        else cout<<"nem prim"; */
    /* for(i=2;i<=szam/2;i++)                //2. megoldás
        if(szam%i==0) osztok++;
    if(osztok==0) cout<<"prim";
        else cout<<"nem prim"; */
    /* for(i=2;i<=sqrt((float)szam);i++)    //3. megoldás
        if(szam%i==0) osztok++;
    if(osztok==0) cout<<"prim";
        else cout<<"nem prim"; */
    /* for(i=2;i<=sqrt((float)szam)&& osztok==0;i++) //4. megoldás
        if(szam%i==0) osztok++;
    if(osztok==0) cout<<"prim";
        else cout<<"nem prim"; */
    /* for(i=2;i<=sqrt((float)szam);i++)    //5. megoldás
        if(szam%i==0) break;
        if(i<=sqrt((float)szam)) cout<<"nem prim"; else cout<<"prim"; */
    /* i=2;
    while(i<=sqrt((float)szam)&& szam%i!=0)    //6. megoldás
        i++;
    if(i<=sqrt((float)szam)) cout<<"nem prim";
        else cout<<"prim"; */
    i=2;
    int x= sqrt((float)szam);
    while(i<=x&& szam%i!=0)                //7. megoldás
        i++;
    if(i<=x) cout<<"nem prim";
        else cout<<"prim";
    cout<<endl;
    system("pause")
    return 0;}
```

2. Kockával dobunk egyszerre 100-szor

a - Töltsünk fel egy tömböt, írassuk ki a képernyőre

b - írassuk ki, hogy az egyes dobásokból mennyi volt

A feladatra bemutatok 3 megoldást.

```
#include<ctime>
# include <iostream>
using namespace std;
int main( )
{ srand((unsigned)time(NULL));
int atlag=0,i,Kocka[100];
//tombfeltöltés
int ered[6]={0};
for( i=0; i<100; i++)
{
Kocka[i]=rand()%6+1;
cout<<Kocka[i]<<" ";
}
// 1. - nagyon gyenge megoldás
for( i=0; i<100; i++)
{
if(Kocka[i]==1) ered[0]++;
if(Kocka[i]==2) ered[1]++;
if(Kocka[i]==3) ered[2]++;
if(Kocka[i]==4) ered[3]++;
if(Kocka[i]==5) ered[4]++;
if(Kocka[i]==6) ered[5]++;
}
// 2. - gyenge megoldás
/*
for( i=0; i<100; i++)
{
switch(Kocka[i])
{
case 1: ered[0]++;break;
case 2: ered[1]++;break;
case 3: ered[2]++;break;
case 4: ered[3]++;break;
case 5: ered[4]++;break;
case 6: ered[5]++;break;
}
}*/
//3. - hatékonyabb megoldás - elágazás helyett indexelt változók
for( i=0; i<100; i++)
```

```

    ered[Kocka[i]-1]++;
// kiíratás
for (i=0; i<6; i++)
    cout<<"\nAz "<<i+1<<"-es dobások száma : "<<ered[i];

    cout<<endl;
system("pause");
return 0;
}

```

3. A jegyek.txt állományban egy harmadikos emeltszintű informatika tanulócsoporthoz adatait tároljuk. Egy csoportba maximum 12-en járhatnak, de nem biztos, hogy mindig betelik a létszám. A fájlban tároljuk a tanulók második év végi és harmadik félévi jegyét és a tanuló keresztnévét. Az állomány egy kicsit "összevisszára sikeredett", első adat a második év végi jegy szóközzel elválasztva a név, majd szóközzel elválasztva a harmadik félévi jegy következik.

PL.: 4 Bela 5

4 Feri 2

3 Lajos 1

Oldja meg az alábbi feladatokat:

1. Olvassa be az állományt, írja ki az adatokat a képernyőre. A beolvasáshoz hozzon létre egy megfelelő adatszerkezetet!
2. Írja ki a képernyőre hányan járnak jelenleg fakultációra!
3. Írja ki a második év végi csoportátlagot!
4. Írja ki melyik félévben volt jobb a csoport tanulmányi eredménye!
5. Döntse el volt-e bukás valamelyik félévben!
6. Ki volt a csoportban a legjobb tanuló a harmadik oszt. félévekor?
7. Rendezze a csoportot névsor szerint növekvő sorrendbe, és írja ki a képernyőre a neveket!
8. Hozzon létre egy új állományt tankor.txt néven. Az eredeti állományban lévő adatok sorrendjét változtassa meg úgy, hogy elől a név utána a második év végi majd a harmadik félévi jegy szerepeljen, majd írja ki az adatokat az állományba!

PL. Bela 4 5

Feri 4 2

A FELADATOKAT FÜGGVÉNYEKKELD OLDJA MEG!

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

struct tanulo
{
    int mv, hf;
    string nev;
};

int Adatbe(tanulo* oszt, string fnev);
float Atlag_masodik(tanulo* oszt, int letszam);
int Jobb_atlag(tanulo* oszt, int letszam);
bool Volt_bukas(tanulo* oszt, int letszam);
string Legjobb(tanulo* oszt, int letszam);
void Rendez(tanulo* oszt, int letszam);
void Kiir(tanulo* oszt, int letszam);

int main()
{
    tanulo osztaly[12];
    string filenev = "jegyek.txt";
    int db = Adatbe(osztaly, filenev);
    cout << "\n" << db << " tanulo jar faktra" << endl;
    cout << "A masodik ev vegi atlag : " << Atlag_masodik(osztaly, db) << endl;
    int felev = Jobb_atlag(osztaly, db);
    if (felev == 0) cout << "Egyforma volt az eredmény a két felevben" << endl;
    else
        if (felev == 2) cout << "A masodik felevben volt jobb az eredmény" << endl;
        else cout << "A harmadik felevben volt jobb az eredmény" << endl;
    if (Volt_bukas(osztaly, db)) cout << "\nVolt bukas" << endl;
    else cout << "\nNem volt bukas" << endl;
    cout << "A csoport legjobb tanuloja a harmadik felevben: " <<
    Legjobb(osztaly, db) << endl;
    cout << "A csoport nevsor szerint abc sorrendben : " << endl;
    Rendez(osztaly, db);
    Kiir(osztaly, db);
    return 0;
}

// 1 – 2. Adatbevitel, létszám meghatározása
int Adatbe(tanulo* oszt, string fnev)
{
    ifstream be(fnev);
    int i = 0;

```

```

    if (be.fail()) { cout << "hiba"; system("pause"); exit(1); }
    while (be >> oszt[i].mv >> oszt[i].nev >> oszt[i].hf)
    {
        cout << oszt[i].mv << " " << oszt[i].nev << " " << oszt[i].hf << endl;
        i++;
    }
    be.close();
    return i;
}

// 3. Írja ki a második év végi csoportátlagot!
float Atlag_masodik(tanulo* oszt, int letszam)
{
    int atl = 0;
    for (int i = 0; i < letszam; i++)
    {
        atl = atl + oszt[i].mv;
    }
    return (float)atl / letszam;
}

//4. Írja ki melyik félévben volt jobb a csoport tanulmányi eredménye!
int Jobb_atlag(tanulo* oszt, int letszam)
{
    float atl_2 = Atlag_masodik(oszt, letszam);
    float atl_3 = 0;
    for (int i = 0; i < letszam; i++)
    {
        atl_3 = atl_3 + oszt[i].hf;
    }
    atl_3 = atl_3 / letszam;

    if (atl_2 > atl_3) return 2;          // a félévek összehasonlítása
    else if (atl_2 == atl_3) return 0;
    else return 3;
}

//5. Döntse el volt-e bukás valamelyik félévben!
bool Volt_bukas(tanulo* oszt, int letszam)
{
    int i=0;
    while((i<letszam)&&(oszt[i].mv!=1)&&(oszt[i].hf!=1))
        i++;
    if (i < letszam) return true;
    else return false;
}

//6. Ki volt a csoportban a legjobb tanuló a harmadik oszt. félévekor?
string Legjobb(tanulo* oszt, int letszam)

```

```

{
    int max=0;
    for(int i=1; i<letszam;i++)
    {
        if(oszt[i].hf>oszt[max].hf) max=i;
    }
    return oszt[max].nev;
}

```

//7. Rendezés

```

void Rendez(tanulo* oszt,int letszam)
{
    string seged[12];
    string temp;
    for(int i=0;i<letszam;i++)
    {
        seged[i]=oszt[i].nev;
    }
    for(int i=0;i<letszam-1;i++)
    {
        for(int j=i+1;j<letszam;j++)
        {
            if(seged[i]>seged[j])
            {
                temp=seged[i];
                seged[i]=seged[j];
                seged[j]=temp;
            }
        }
    }
    for(int i=0;i<letszam;i++)
        cout<<seged[i]<<endl;
}

```

//8. Hozzon létre egy új állományt tankor.txt néven. Az eredeti állományban lévő adatok sorrendjét változtassa meg úgy, hogy elől a név utána a második év végi majd a harmadik félévi jegy szerepeljen, majd írja ki az adatokat az állományba!

```

void Kiir(tanulo* oszt,int letszam)
{
    ofstream ki("tankor.txt");
    for (int i = 0; i < letszam; i++)
    {
        ki << oszt[i].nev << " " << oszt[i].mv << " " << oszt[i].hf << endl;
    }
    ki.close();
    cout << "\nA tankor.txt fájl létrehozva es az adatok kiirva. ";
}

```